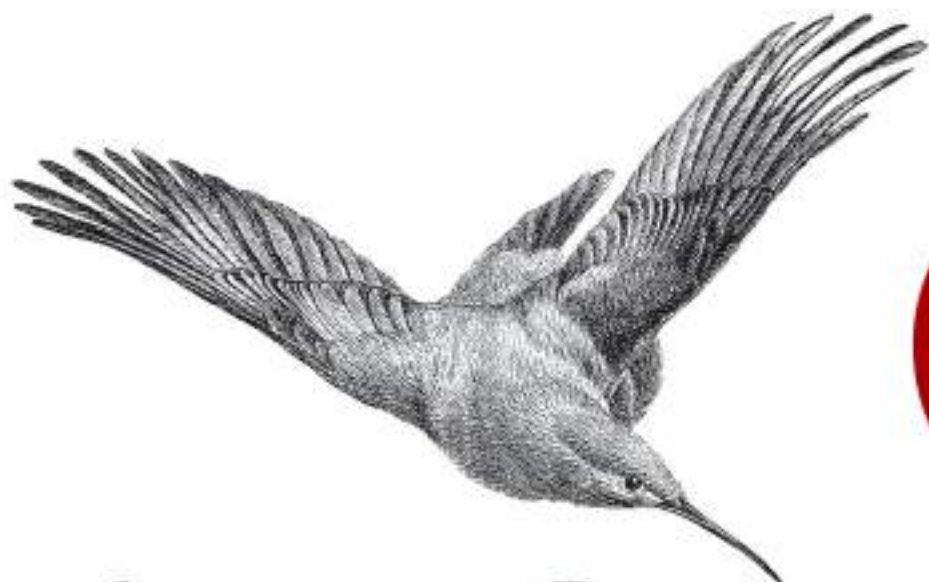


O'REILLY®



Vibe Coding

The Future of Programming

Leverage Your
Experience in the
Age of AI



Addy Osmani

Vibe Coding: The Future of Programming

Leveraging Your Experience in the Age of AI-Assisted Coding

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Addy Osmani

O'REILLY®

Vibe Coding: The Future of Programming

by Addy Osmani

Copyright © 2025 Addy Osmani. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Louise Corrigan

Development Editor: Sarah Grey

Production Editor: Katherine Tozer

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

August 2025: First Edition

Revision History for the Early Release

- 2025-04-17: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341634756> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Vibe Coding: The Future of Programming*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the

publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-63470-1

Brief Table of Contents (*Not Yet Final*)

Chapter 1: The Vibe Shift: Programming with Intent (unavailable)

Chapter 2: The Art of the Prompt: Communicating Effectively with AI (unavailable)

Chapter 3: The 70% Problem: Hard Truths About AI-Assisted Coding (available)

Chapter 4: Beyond the 70%: Maximizing Human Contribution (available)

Chapter 5: Understanding Generated Code: Review, Refine, Own (unavailable)

Chapter 6: AI-Driven Prototyping: Tools and Rechniques (unavailable)

Chapter 7: Building Web Applications with AI (unavailable)

Chapter 8: Security and Reliability with AI-Generated Code (unavailable)

Chapter 9: Ethical Implications of Vibe Coding (unavailable)

Chapter 10: The Unbundling of the Programmer: Personal Software (unavailable)

Chapter 11: Beyond Code Generation: AI's Expanding Role (unavailable)

Chapter 12: The Vibe Coder's Toolkit: Advanced Techniques (unavailable)

Chapter 1. The 70% Problem: AI-Assisted Workflows That Actually Work

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at sgrey@oreilly.com.

AI-based coding tools are astonishingly good at certain tasks.¹ They excel at producing boilerplate, writing routine functions, and getting projects *most of the way* to completion. In fact, many developers find that an AI assistant can implement an initial solution that covers roughly 70% of the requirements.

A [tweet](#) from Peter Yang perfectly captures what I’ve been observing in the field:

Honest reflections from coding with AI so far as a non-engineer:

It can get you 70% of the way there, but that last 30% is frustrating. It keeps taking one step forward and two steps backward with new bugs, issues, etc.

If I knew how the code worked I could probably fix it myself. But since I don’t, I question if I’m actually learning that much.

Non-engineers using AI for coding find themselves hitting a frustrating wall. They can get 70% of the way there surprisingly quickly, but that final 30% becomes an exercise in diminishing returns.

This “70% problem” reveals something crucial about the current state of AI-assisted development. The initial progress feels magical - you can describe what you want, and AI tools like v0 or Bolt will generate a working prototype that looks impressive. But then reality sets in.

The 70% is often the straightforward, patterned part of the work – the kind of code that follows well-trod paths or common frameworks. As one *Hacker News* commenter **observed**, AI is superb at handling the “accidental complexity” of software (the repetitive, mechanical stuff) while the “essential complexity” – understanding and managing the inherent complexity of a problem – remains on human shoulders. In Fred Brooks’ classic terms, AI tackles the incidental, but not the intrinsic, difficulties of development.

Where do these tools struggle? Experienced engineers consistently report a “last mile” gap. AI can generate a plausible solution, but the final 30% – covering edge cases, refining the architecture, and ensuring maintainability – “needs serious human expertise”.

For example, an AI might give you a function that technically works for the basic scenario, but it won’t automatically account for unusual inputs, race conditions, performance constraints, or future requirements unless explicitly told. AI can get you most of the way there, but that final crucial 30% (edge cases, keeping things maintainable, and solid architecture) needs serious human expertise.

Moreover, AI has a known tendency to generate convincing but incorrect output. It may introduce subtle bugs or “hallucinate” nonexistent functions and libraries. Steve Yegge **wryly likens** today’s LLMs to “wildly productive junior developers” – incredibly fast and enthusiastic, but “potentially whacked out on mind-altering drugs,” prone to concocting crazy or unworkable approaches.

In Yegge’s **words**, an LLM can spew out code that looks polished at first glance, yet if a less experienced developer naively says “Looks good to me!” and runs with it, hilarity (or disaster) ensues in the following weeks. The AI doesn’t truly *understand* the problem; it stitches together patterns that *usually* make sense. Only a human can discern whether a seemingly fine solution hides long-term landmines. Simon Willison **echoed this** after seeing an AI propose a bewitchingly clever design that *only a senior engineer with deep understanding of the problem* could recognize as flawed. The lesson: AI’s confidence far

exceeds its reliability.

Crucially, current AIs **do not** create fundamentally new abstractions or strategies beyond their training data. They won't invent a novel algorithm or an innovative architecture for you – they remix what's known. They also won't take responsibility for decisions. As one engineer noted, “AIs don't have ‘better ideas’ than what their training data contains. They don't take responsibility for their work.”

All of this means that the creative and analytical thinking – deciding *what* to build, *how* to structure it, and *why* – firmly remains a human domain. In summary, AI is a force multiplier for developers, handling the repetitive 70% and giving us a “turbo boost” in productivity. But it is *not* a silver bullet that can replace human judgment. The remaining 30% of software engineering – the hard parts – still requires skills that only trained, thoughtful developers can bring. Those are the durable skills to focus on, and Chapter 6 is dedicated to them. As one **discussion** put it: “AI is a powerful tool, but it's not a magic bullet... human judgment and good software engineering practices are still essential.”

I've observed two distinct patterns in how teams are leveraging AI for development. Let's call them the “bootstrappers” and the “iterators.” Both are helping engineers (and even non-technical users) reduce the gap from idea to execution (or MVP).

First, there are the *bootstrappers*, who are generally taking a new project from zero to MVP. Tools like Bolt, v0, and screenshot-to-code AI are revolutionizing how these teams bootstrap new projects. These teams typically:

- Start with a design or rough concept
- Use AI to generate a complete initial codebase
- Get a working prototype in hours or days instead of weeks
- Focus on rapid validation and iteration

The results can be impressive. I recently watched a solo developer use Bolt to turn a Figma design into a working web app in next to no time. It wasn't production-ready, but it was good enough to get very initial user feedback.

The second camp, the *iterators*, use tools like Cursor, Cline, Copilot, and

WindSurf for their daily development workflow. This is less flashy but potentially more transformative. These developers are:

- Using AI for code completion and suggestions
- Leveraging AI for complex refactoring tasks
- Generating tests and documentation
- Using AI as a “pair programmer” for problem-solving

But here’s the catch: while both approaches can dramatically accelerate development, they come with hidden costs that aren’t immediately obvious.

When you watch a senior engineer work with AI tools like Cursor or Copilot, it looks like magic. They can scaffold entire features in minutes, complete with tests and documentation. But watch carefully, and you’ll notice something crucial: They’re not just accepting what the AI suggests. They’re constantly refactoring the generated code into smaller, focused modules. They’re adding comprehensive error handling and edge-case handling the AI missed, strengthening its type definitions and interfaces, and questioning its architectural decisions. In other words, they’re applying years of hard-won engineering wisdom to shape and constrain the AI’s output. The AI is accelerating their implementation, but their expertise is what keeps the code maintainable.

Common Failure Patterns

Junior engineers often miss these crucial steps. They accept the AI’s output more readily, leading to what I call “house of cards code” – it looks complete but collapses under real-world pressure.

Two Steps Back

What typically happens next follows a predictable pattern I call the “two steps back” pattern (shown in [Figure 1-1](#)):

- You try to fix a small bug.
- The AI suggests a change that seems reasonable.

- This fix breaks something else.
- You ask AI to fix the new issue.
- This creates two more problems.
- Rinse and repeat.

Two Steps Back

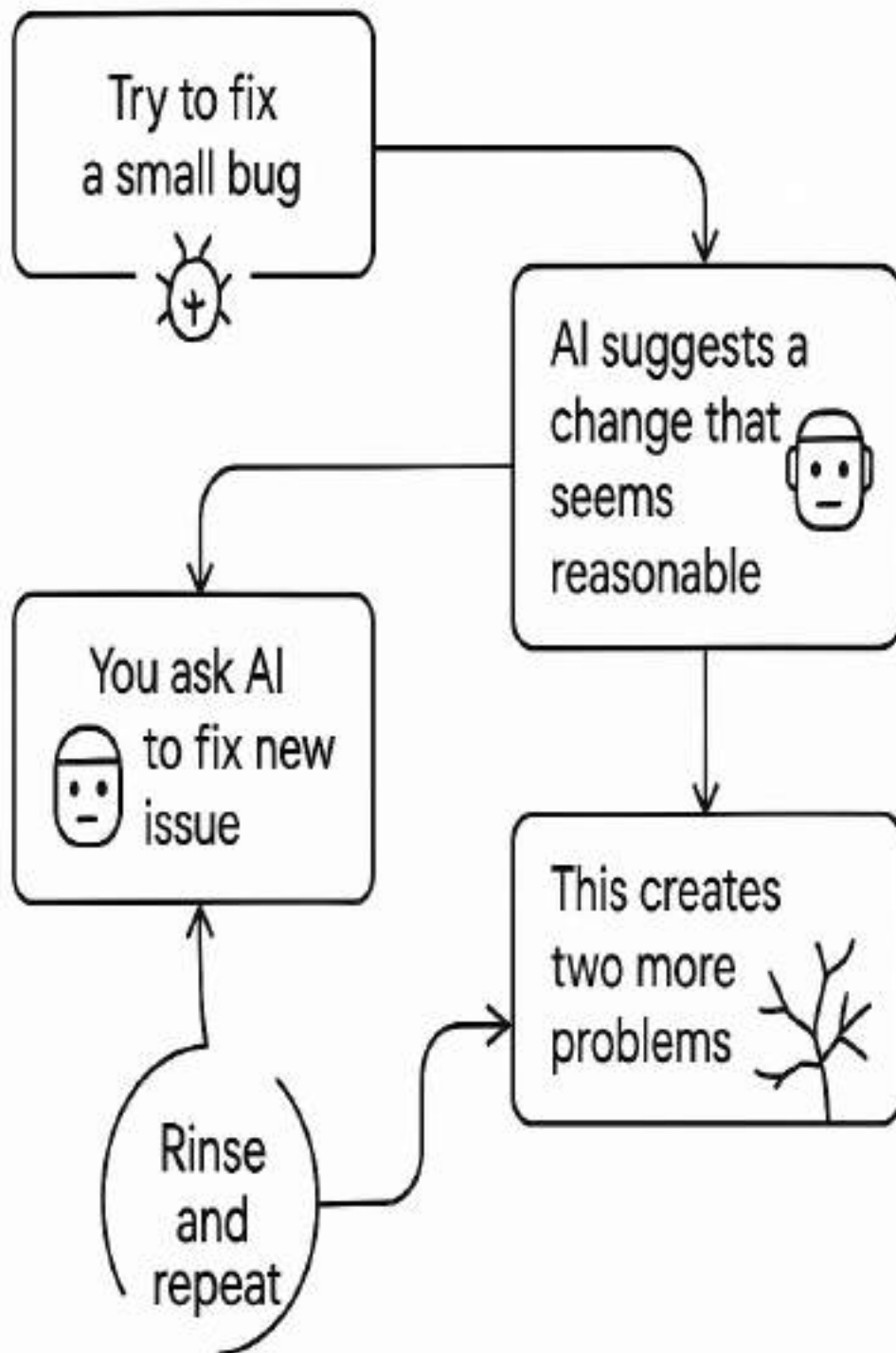


Figure 1-1. The “two steps back” antipattern

This cycle is particularly painful for non-engineers because they lack the mental models to understand what’s actually going wrong. When an experienced developer encounters a bug, they can reason about potential causes and solutions based on years of pattern recognition. Without this background, you’re essentially playing whack-a-mole with code you don’t fully understand. This is the “knowledge paradox” I mentioned back in this book’s preface: Senior engineers and developers use AI to accelerate what they already know how to do, while juniors try to use it to learn *what* to do.

This cycle is particularly painful for nonengineers using AI in a “bootstrapper” pattern, because they lack the mental models needed to address these issues building their MVP. However, even experienced “iterators” can fall into this whack-a-mole trap if they overly rely on AI suggestions without deep validation.

There’s a deeper issue here: The very thing that makes AI coding tools accessible to nonengineers—their ability to handle complexity on your behalf—can actually impede learning. When code just “appears” without you understanding the underlying principles, you don’t develop debugging skills. You miss learning fundamental patterns. You can’t reason about architectural decisions, and so you struggle to maintain and evolve the code. This creates a dependency where you need to keep going back to the AI model to fix issues, rather than developing the expertise to handle them yourself.

This dependency risk may deepen with the rise of more autonomous ‘agentic’ AI systems. These agents don’t just suggest code snippets: they can plan, execute, and iterate on entire development tasks with minimal human input. As I write this in early 2025, tools like Cline, Devin AI, and Claude Code are already demonstrating the ability to autonomously debug, test, and even deploy code. While this promises efficiency, it also introduces new challenges. Without a solid understanding of the underlying processes, users may find themselves increasingly reliant on these agents, unable to intervene effectively when things go awry.

Moreover, as agentic AI systems become more integrated into development workflows, the potential for cascading errors increases. An autonomous agent might make a series of decisions that, while individually sound, collectively lead

the project in an unintended direction. Without the expertise to audit and correct these decisions, users risk building on unstable foundations.

In essence, while agentic AI offers powerful tools for software development, it also amplifies the importance of foundational knowledge. To harness these tools effectively and responsibly, users must cultivate a deep understanding of software principles, ensuring they remain in control of the development process rather than becoming passive observers.

The Demo-Quality Trap

It's becoming a pattern: Teams use AI to rapidly build impressive demos. The happy path works beautifully. Investors and social networks are wowed. But when real users start clicking around? That's when things fall apart.

I've seen this firsthand: error messages that make no sense to normal users, edge cases that crash the application, confusing UI states that never got cleaned up, accessibility completely overlooked, performance issues on slower devices. These aren't just low-priority bugs—they're the difference between software people tolerate and software people love.

Creating truly self-serve software—the kind where users never need to contact support—requires a different mindset, one that's all about the lost art of polish. You need to be obsessing over error messages, testing on slow connections and with real, nontechnical users, making features discoverable, and handling every edge case gracefully. This kind of attention to detail (perhaps) can't be AI-generated. It comes from empathy, experience, and caring deeply about craft.

What Actually Works: Practical Workflow Patterns

Before we dive into coding in Part II of this book, we need to talk about modern development practices and how AI-assisted coding fits within a team workflow. Software development is more than writing code, after all – it's a whole workflow that includes planning, collaboration, testing, deployment, and maintenance. And vibe coding isn't a standalone novelty – it can be woven into agile methodologies and DevOps practices, augmenting the team's productivity

while preserving quality and reliability.

In this section, we'll explore how team members can collectively use vibe coding tools without stepping on each other's toes, how to balance AI suggestions with human insight, and how continuous integration/continuous delivery (CI/CD) pipelines can incorporate AI or adapt to AI-generated code. I'll also touch on important considerations like version-control strategies.

After observing dozens of teams, here are three patterns I've seen work consistently in both solo and team workflows:

- *AI as first drafter*, where the AI model generates the initial code and developers then refine, refactor, and test it.
- *AI as pair programmer*, where developer and AI are in constant conversation, with tight feedback loops, frequent code review, and minimal context provided
- *AI as validator*, where developers still write the initial code and then use AI to validate, test, and improve it

In this section, I'll walk you through each pattern in turn, discussing workflows and tips for success.

AI as First Drafter

It's important to make sure everyone on the team is on the same page before you ask your AI model to begin drafting any code. Communication is key so that developers don't ask their AI assistants to do redundant tasks or generate conflicting implementations.

In daily stand-ups (a staple of agile workflows), it's worth discussing not just what you're working on, but also if you plan to use AI for certain tasks. For example, two developers might be working on different features that both involve a utility function for date formatting – if both ask the AI to create a `formatDate` helper, you might end up with two similar functions.

Coordinating upfront (“I’ll generate a date utility we can both use”) can prevent duplication.

Teams that successfully integrate AI tools often start by agreeing on coding

standards and prompting practices. For example, the team might decide on a consistent style (linting rules, project conventions) and even feed those guidelines into their AI tools (some assistants allow providing style preferences or example code to steer outputs). As [Codacy's blog notes](#), by familiarizing the AI with the team's coding standards, you get generated code that is more uniform and easier for everyone to work with. On a practical level, this could mean having a section in your project README for "AI Usage Tips," where you note things like "We use functional components only" or "Prefer using Fetch API over Axios," which developers can keep in mind when prompting AI.

Another practice is to use your tools' *collaboration features* if available. Some AI-assisted IDEs allow users to share their AI sessions, or at least the prompts they use. If Developer A got a great result with a prompt for a complex component, sharing that prompt with Developer B (perhaps via the issue tracker or a team chat) can save time and ensure consistency.

As for using version control, the fundamentals remain – with a twist. Using Git (or another version control system) is non-negotiable in modern development, and that doesn't change with vibe coding. In fact, version control becomes even more crucial when AI is generating code rapidly. Commits act as the safety net to catch AI missteps; if an AI-generated change breaks something, you can revert to a previous commit.

One strategy is to commit more frequently when using AI assistance. Each time the AI produces a significant chunk of code (like generating a feature or doing some major refactoring) that you accept, consider making a commit with a clear message. Frequent commits ensure that if you need to bisect issues or undo a portion of AI-introduced code, the history is granular enough.

Also, try to isolate different AI-introduced changes. If you let the AI make many changes across different areas and commit them all together, it's harder to disentangle if something goes wrong. For example, if you use an agent to optimize performance and it also tweaks some UI texts, commit those separately. (Your two commit messages might be "Optimize list rendering performance [AI-assisted]" and "Update UI copy for workout completion message [AI-assisted]"). Descriptive commit messages are important; some teams even tag commits that had heavy AI involvement, just for traceability. It's not about blame, but about understanding the origin of code – a commit tagged with "[AI]" might signal to a

reviewer that the code could use an extra-thorough review for edge cases.

Essentially, the team should treat AI usage as a normal part of the development conversation: share experiences, successful techniques, and warnings about what not to do (like “Copilot suggests using an outdated library for X, be careful with that”).

Review and refinement are crucial to this pattern. Developers should manually review and refactor the code for modularity, add comprehensive error handling, write thorough tests, and document key decisions as they refine the code. The next chapter goes into detail about these processes.

AI as Pair Programmer

Traditional pair programming involves two humans collaborating at one workstation. With the advent of AI, a hybrid approach has emerged: one human developer working alongside an AI assistant. This setup can be particularly effective, offering a blend of human intuition and machine efficiency.

In a human-AI pairing, the developer interacts with the AI to generate code suggestions, while also reviewing and refining the output. This dynamic allows the human to leverage the AI’s speed in handling repetitive tasks, such as writing boilerplate code or generating test cases, while maintaining oversight to ensure code quality and relevance.

For instance, when integrating a new library, a developer might prompt the AI to draft the initial integration code. The developer then reviews the AI’s suggestions, cross-referencing with official documentation to verify accuracy. This process not only accelerates development but also facilitates knowledge acquisition, as the developer engages deeply with both the AI’s output and the library’s intricacies.

Let’s compare this to traditional human-human pair programming:

- *Human-AI pairing* offers rapid code generation and can handle mundane tasks efficiently. It’s particularly beneficial for solo developers or when team resources are limited.
- *Human-human pairing* excels in complex problem-solving scenarios, where nuanced understanding and collaborative brainstorming are

essential. It fosters shared ownership and collective code comprehension.

Both approaches have their merits, and your choice between them can be guided by the project's complexity, resource availability, and the specific goals of the development process.

Best Practices for AI Pair Programming

To maximize the benefits of AI-assisted development, consider the following practices:

Initiate new AI sessions for distinct tasks

This helps maintain context clarity and ensures the AI's suggestions are relevant to the specific task at hand.

Keep prompts focused and concise

Providing clear and specific instructions enhances the quality of the AI's output.

Review and commit changes frequently

Regularly integrating and testing AI-generated code helps catch issues early and maintains project momentum.

Maintain tight feedback loops

Continuously assess the AI's contributions, providing corrections or refinements as needed to guide its learning and improve future suggestions.

AI as Validator

Beyond code generation, AI can serve as a valuable validator, assisting in code review and quality assurance. AI tools can analyze code for potential bugs, security vulnerabilities, and adherence to best practices. For example, platforms like DeepCode and Snyk's AI-powered code checker can identify issues such as missing input sanitization or insecure configurations, providing actionable insights directly within the development environment. Platforms such as Qodo

and TestGPT can automatically generate test cases, ensuring broader coverage and reducing manual effort. And many AI tools can assist in monitoring application performance, detecting anomalies that might indicate underlying issues.

By integrating AI validators into the development workflow, teams can enhance code quality, reduce the likelihood of defects, and ensure compliance with security standards. This proactive approach to validation complements human oversight, leading to more robust and reliable software. These tools enhance the efficiency and effectiveness of the QA process by handling repetitive and time-consuming tasks, allowing human testers to focus on more complex and nuanced aspects of quality assurance.

Incorporating AI into the development process, whether as a pair programmer or validator, offers opportunities to enhance productivity and code quality. By thoughtfully integrating these tools, developers can harness the strengths of both human and artificial intelligence.

To maximize the benefits of both AI and human capabilities in QA, I recommend a few best practices:

- Use AI for initial assessments and preliminary scans to identify obvious issues.
- Prioritize human review for critical areas, such as complex functionalities, user experience, and areas where AI may have limitations.
- Foster an environment of continuous collaboration, where AI tools and human testers work in tandem, with ongoing feedback loops to improve both AI performance and human decision-making.

The Golden Rules of Vibe Coding

Be specific and clear about what you want

Clearly articulate your requirements, tasks, and outcomes when interacting with AI. Precise prompts yield precise results.

Always validate AI output against your intent

AI-generated code must always be checked against your original goal. Verify functionality, logic, and relevance before accepting.

Treat AI as a junior developer (with supervision)

Consider AI outputs as drafts that require your careful oversight. Provide feedback, refine, and ensure quality and correctness.

Use AI to expand your capabilities, not replace your thinking

Leverage AI to automate routine or complex tasks, but always remain actively engaged in problem-solving and decision-making.

Coordinate upfront among the team before generating code

Align with your team on AI usage standards, code expectations, and practices before starting AI-driven development.

Treat AI usage as a normal part of the development conversation

Regularly discuss AI experiences, techniques, successes, and pitfalls with your team. Normalize AI as another tool for collective improvement.

Isolate AI changes in Git by doing separate commits

Clearly identify and separate AI-generated changes within version control to simplify reviews, rollbacks, and tracking.

Ensure that all code, whether human or AI-written, undergoes code review

Maintain consistent standards by subjecting all contributions to the same rigorous review processes, enhancing code quality and team understanding.

Don't merge code you don't understand

Never integrate AI-generated code unless you thoroughly comprehend its functionality and implications. Understanding is critical to maintainability and security.

Prioritize documentation, comments, and ADRs

Clearly document the rationale, functionality, and context for AI-generated code. Good documentation ensures long-term clarity and reduces future technical debt.

Share and reuse effective prompts

Document prompts that lead to high-quality AI outputs. Maintain a repository of proven prompts to streamline future interactions and enhance consistency.

Regularly reflect and iterate

Periodically review and refine your AI development workflow. Use insights from past experiences to continuously enhance your team's approach.

By adhering to these golden rules, your team can harness AI effectively, enhancing productivity while maintaining clarity, quality, and control.

¹ This chapter is based on an essay originally published on my Substack newsletter, *Elevate with Addy Osmani*, “[The 70% Problem: Hard Truths about AI-Assisted Coding](#),” December 4, 2024.